PHARMAMOBIL

Informe técnico del proyecto

Copia preparada para Gabriel Vasquez Toro

Estudiante	Docente	Fecha
Gabriel Vasquez Toro	David Reyna	10 de noviembre de 2026

Repositorio

https://github.com/gabrielvastoro2005-byte/FarmaMobile | rama main

Resumen

Esta copia reúne el código multiplataforma de PharmaMobile y los avances documentados de Clean Architecture + MVVM, acceso JWT a PharmaSoft, persistencia offline-first, capacidades nativas y preparación de distribución Android. La copia se identificó para Gabriel y fue verificada en Windows con compilación Android y pruebas Android Host.

Estado de entrega

El código se publica en el repositorio indicado. El informe distingue las verificaciones ejecutadas de las tareas que dependen de Firebase, macOS/Xcode o dispositivos físicos. Los builds y certificados del proyecto anterior se dejaron fuera; no se atribuyen a Gabriel.

1. Arquitectura y funcionalidades

La app usa Kotlin Multiplatform y Compose Multiplatform. `shared/src/commonMain` concentra dominio, casos de uso, repositorios, ViewModels, pantallas y navegación. Koin conecta los módulos por plataforma. SQLDelight gestiona almacenamiento local; Ktor comunica la app con PharmaSoft y con el catálogo de práctica. Los source sets Android/iOS alojan los puentes nativos.

Sesión	Funcionalidad incluida en esta copia
9 - Capacidades nativas	Formato de soles con expect/actual y compartir producto mediante un contrato de dominio y adaptadores nativos.
10 - Autenticación	Inicio, recuperación y cierre de sesión; JWT; almacenamiento seguro; CRUD remoto autenticado de productos y clientes.
11 - Offline-first	Persistencia de inventario local, altas pendientes, sincronización con PharmaSoft y conservación de registros sin conexión.
13 - Integrador	Contadores de sincronización, errores/intententos persistentes, política de reintentos, permisos por capas y puerto de telemetría.
14 - Builds	Configuración versionada, R8 y lectura de firma Release desde configuración local ignorada por Git.

Regla arquitectónica

La interfaz representa estado y dispara acciones. Las reglas de negocio viven en el dominio y los ViewModels. Los adaptadores de persistencia, red, permisos y telemetría implementan contratos para que las capas superiores no dependan de APIs concretas.

Autenticación

La sesión usa rutas `/api/v1/auth/login` y `/api/v1/auth/me`; los clientes HTTP restringidos y públicos están separados. El token se guarda con almacenamiento seguro en Android/iOS. Los permisos del usuario controlan las acciones remotas.

2. Persistencia, sincronización y capacidades nativas

La tabla local de productos mantiene estado pendiente, contador de intentos y último error; la migración `1.sqm` añade los campos nuevos. La lista de la interfaz procede de SQLDelight. Los productos nuevos se guardan primero de forma local y luego se intentan sincronizar.

El caso de uso distingue éxito, rechazo permanente y error transitorio. Limita el proceso a cinco intentos y aplica esperas exponenciales de 1, 2, 4, 8 y hasta 16 segundos cuando el adaptador indica que repetir el envío es seguro. Errores de resultado ambiguo no se reenvían automáticamente para reducir riesgo de duplicados. La pantalla muestra pendientes, fallidos, fecha relativa de última sincronización y detalle del error.

Permisos

El dominio define cámara, ubicación y notificaciones, con estados concedido, denegado y denegado permanentemente. Android consulta/solicita mediante un puente asociado a Activity, muestra el rationale y abre Ajustes. El manifiesto declara cámara, ubicación precisa y notificaciones. iOS incluye textos de uso de cámara y ubicación, pero su puente de solicitud nativa sigue pendiente y retorna NoDisponible.

Telemetría

El código usa un puerto `Telemetria` y un adaptador Android basado en Crashlytics que filtra claves sensibles. El proyecto Firebase real y `androidApp/google-services.json` no están configurados en esta copia, por lo que no se declara recepción de eventos. iOS requiere configuración y frameworks propios.

Pruebas de sincronización existentes

Los tests Android Host cubren alta local, reconexión, éxito remoto, reintento seguro, rechazo permanente, conservación de pendientes, ids locales/remotos y ausencia de duplicados.

3. Verificación y límites de entrega

Comprobación	Resultado	Alcance
:shared:testAndroidHostTest	Correcto	Pruebas host/JVM completadas en esta copia.
:androidApp:compileDebugKotlin	Correcto	Código Android y módulo compartido compilan.
Build Release firmado para Gabriel	Pendiente	Se excluyó el keystore del proyecto original; falta crear una clave propia y configurar `keystore.properties`.
Dos teléfonos Android físicos	Pendiente	ADB no detectó dispositivos desde este entorno; no se inventan modelos ni resultados.
iOS Archive/IPA	Pendiente	Requiere macOS, Xcode y certificados/profiles Apple.
Firebase Crashlytics	Pendiente	Falta configuración del proyecto Firebase y prueba de recepción.

Comandos reproducibles ejecutados

```
./gradlew.bat :shared:testAndroidHostTest :androidApp:compileDebugKotlin
```

Próximos pasos

1. Preparar y respaldar un keystore de Gabriel fuera del repositorio; configurar `keystore.properties` y generar AAB/APK Release nuevos.
2. Ejecutar la matriz en al menos dos teléfonos Android y registrar modelo, versión, resultados y defectos reales en `docs/MATRIZ_PRUEBAS_DISPOSITIVOS.md`.
3. En macOS/Xcode, validar iOS y producir Archive/IPA; implementar el puente nativo de permisos iOS.
4. Configurar Firebase de esta app y verificar un evento de prueba sin datos personales.

Conclusión

La copia independiente para Gabriel está publicada en la rama `main` con su propio historial Git. Las pruebas automatizadas y compilación Android indicadas pasaron. La entrega se debe considerar parcialmente validada hasta completar firma propia, Firebase, iOS y pruebas físicas.